Introduction, Objectives & Contributions

A visual workbench to build, test and *understand* algorithmic trading strategies.

Introduction

ATLAS — Algorithmic Trading Lab and Analysis System — is a human-centered web app on top of the QuantConnect LEAN engine.

One connected flow replaces scattered tools: Create → Configure → Backtest → Explore → Compare → Understand.

Course project, AI511 – HCI & Visualization, School of AI & Data Engineering, IIT Ropar.

CODE

`github.com/rishavrai563/A.T.L.A.S.`

Objectives

- Make strategy creation easy — **Python or visual blocks**
- Make backtests simple to configure, with **clear progress**
- Show **how parameter changes** move the results
- Compare runs **with their configuration** in view
- Cut avoidable errors; improve **accessibility** from user testing

Rishav Kumar · 2024AIB1014

- **Backend & engine** — ASP.NET Core API, LEAN Docker runner, result parser, async job manager
- **Data** — market-data pipeline (Yahoo/Upstox → CSV), dynamic parameter injection
- **Workflow UI** — Strategy Builder, Parameters, live backtest progress
- **Usability fixes** — DocTooltip glossary, Reset confirmation, API docs, user-flow diagram

55 COMMITS

Ravikant Sharma · 2024AIB1013

- **Frontend foundation** — React app shell, navigation and layout
- **Accounts & profile** — login UI, user profile and portfolio page
- **Theming** — light/dark toggle; dark styling for Compare, Parameters, Profile
- **Documentation** — README, setup, architecture, tech stack, user flow

23 COMMITS

Goals

Trading tools help you run strategies — not *understand* them.

Problem Statement

Algorithmic trading platforms are powerful, but users juggle **code, charts, parameters and metrics** across disconnected screens.

The result: people can **run** a backtest, yet struggle to see **why** a result changed or which configuration produced it.

Our Lab 7 study confirmed it — a gap between performing an action and understanding its consequences.

Gaps in existing tools

Gulf of execution — TradingView strategies need Pine Script, so intent must be translated into code

Cognitive load — QuantConnect spans research, coding, backtesting and analysis stages

Exploration — changing logic means re-editing code; comparing variants is effortful

Design goals

- Interactive strategy building
- Visual exploration
- Informed comparison
- Understanding, in place

Personas: beginner trader · quant student · quant researcher · strategy analyst.

Success targets · fixed before testing

SC1 · INDEPENDENT COMPLETION
≥ 4 of 5 users, per task

SC2 · COMPLETION TIME
≤ 240 s T1 · ≤ 300 s T2

SC3 · ERRORS
≤ 1.0 mean per task

SC4 · UNDERSTANDING
≥ 4.0 / 5 questionnaire mean

T1 STRATEGY CREATION & BACKTESTING · T2 EXPLORATION & COMPARISON

SOURCES: LAB 4, LAB 5, LAB 9

Methodology & Technical Implementation

User research first, then a *full-stack* build on the LEAN engine.

Methodology

LAB 4
Study TradingView & QuantConnect; find HCI gaps

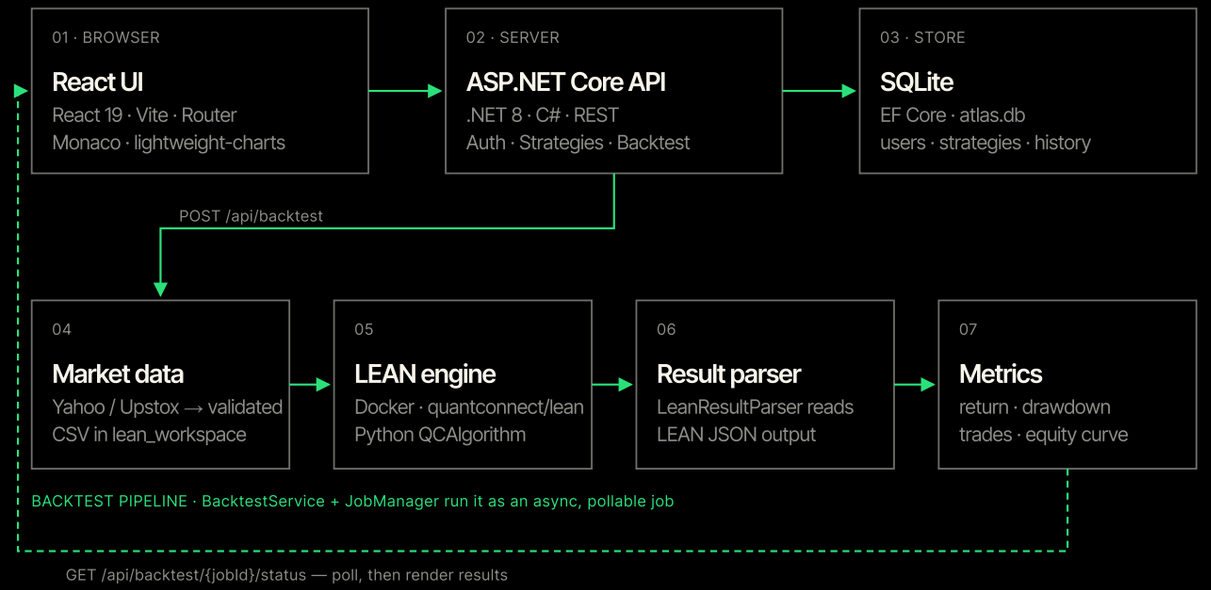
LAB 5
Design personas, key tasks, low-fi screens, user flow

LAB 7
Evaluate 5 users + heuristics + accessibility

LAB 8
Improve tooltips, Compare table, Reset modal, Monaco, contrast

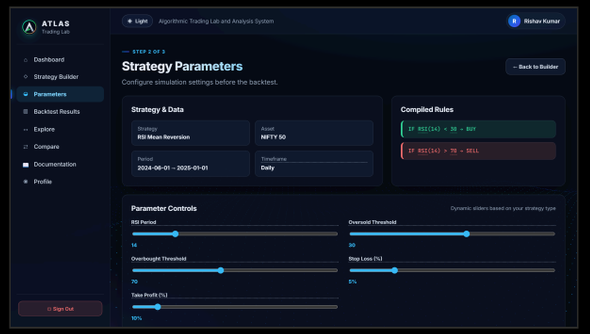
LAB 9
Validate 5 new users against pre-set targets

System architecture



≈10.7K LINES · 78 COMMITS · BEARER-TOKEN AUTH · SALTED PBKDF2 · DARK/LIGHT THEME

The workbench



02 · PARAMETERS — SLIDERS + COMPILED RULES

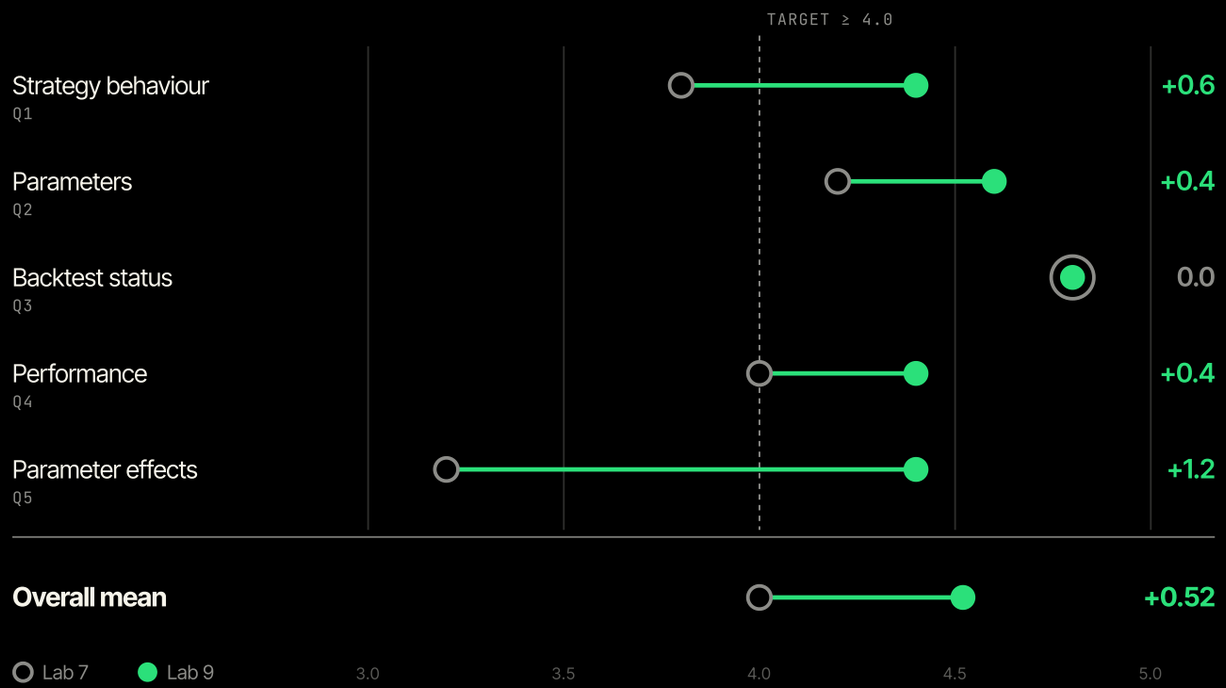


03 · RESULTS — METRICS, CHART, TRADES

Results & Evaluation

All four targets *met* — understanding rose from 4.0 to *4.52*/5.

Questionnaire · Lab 7 vs Lab 9 · mean rating, 1–5



Targets vs. results · N = 5 new users

SC1 · INDEPENDENT COMPLETION
4/5 T1 · 5/5 T2 — MET

SC2 · MEAN TIME
202 s T1 · 248 s T2 — MET

SC3 · MEAN ERRORS
0.6 T1 · 0.6 T2 — MET

SC4 · UNDERSTANDING
4.52 / 5 (25 ratings) — MET

Lab 7 issues re-checked

Fixed — parameters reach LEAN, Compare config table, Reset confirm, tooltips, Monaco editor, Estimated label, contrast, text size — 8

Partly — accessible labels — 1

Open — trade rows not keyboard-operable — 1

Largest gain: parameter effects (Q5) +1.2. Two different user groups — a descriptive comparison, not proof of cause. Status per the Lab 9 implementation guide.

[PLACEHOLDER] Lighthouse accessibility score

[PLACEHOLDER] Testing date & build commit

T1 STRATEGY CREATION & BACKTESTING · T2 EXPLORATION · SC = SUCCESS CRITERION

SOURCE: LAB 9 REPORT (TABLES 3-6, 9)

Findings, Insights & Key Learnings

Closing the gap between *doing* and understanding.

What worked

Side-by-side Compare with a "Changed?" highlight — Q5 up 3.2 → 4.4 (N02, N01)

In-place tooltips for RSI, Sharpe, Drawdown reassured novices (N01, N04)

Reset confirmation protected configuration (N04)

Date validation caught an inverted range (N03)

Insights & limits

Run Backtest was easy to miss — N04 needed a hint after 65 s

Compare was hard to find — N01 first edited parameters in the builder

Trade table — N03 tried arrow keys before using the mouse

Scope — 5 users, different from Lab 7; 5 custom questions, not SUS

Key learnings

Recognition over recall — show the configuration next to every result

Explain jargon in place, not in a separate manual

Guard destructive actions; validate inputs before they run

Finishing a task ≠ accessible — test keyboard use, not only success

Pre-set targets keep the evaluation honest

Next steps

Make **Run Backtest** more prominent

Keyboard-operable **trade table** and chart markers `BUILT, RE-TEST`

Audit **accessible names** on every page

Measure **contrast** in light and dark themes

Verify **parameter propagation** to LEAN end to end

`[PLACEHOLDER] Live demo / Q&A slot`